

▼ pass 1

Read the **Abstract, Introduction (Section 1)**, the setup in **Section 2 up through 2.1 (high level)**, then skim **Sections 3–5** for the punchlines

▼ mental notes

- what are MPNNs, what are graph transformers (is this attentional flavour?)
- why can MPNNs leverage graph sparsity?
- current literature says oversquashing only happens in long range tasks, this paper says it can happen in short range tasks.
oversquashing is the phenomenon where node-feature dimension is the bottleneck for learning GNNs, and so oversquashing necessitates larger node feature dimension. long range tasks result in oversquashing. but they show that short range tasks do too (i.e. learning the correct GNN is limited by node-feature dimension). they say that this is the case even though the underlying graph is well connected (in long range tasks, low connectivity causes oversquashing).
- remember low connectivity, vanishing gradients are the 2 phenomenon in the literature which necessitates the need for larger node-feature dimension to train GNNs well.
- here there is high connectivity AND now vanishing gradients.
- so they basically propose another phenomenon in which we require larger node-feature dimension to train GNNs well.
- basically take as definition oversquashing: require larger node-feature dimension to train GNNs well
- is the bottleneck effect: require larger node-feature dimension to train GNNs well?
- they show popular synthetic long-range tasks suffer from vanishing gradients but NOT bottleneck effect.

- so they claim that these are 2 phenomenon that were mixed together in the literature.
 - for all of these problems: bottleneck effects, vanishing gradients. MPNNS (Even when using virtual nodes) fails compared to transformers.
-

Section 1

Set up the problem the authors care about

- We investigate MPNN flavour of GNNs
- A limitation of MPNNs are oversquashing, and this motivates our study.
- The standard story of oversquashing from the literature is that it is a phenomenon that arises during long range tasks.
- A long range task, is a task which requires distant nodes to share information, in order to be solved.
- Long range tasks may be hard for message passing in general because information sharing for MPNNs is local. Information from distant nodes must move through many steps, and due to the fixed dimensionality of node feature vectors, this information may get compressed along the way.
- If a graph struggles with long range tasks, two approaches are most common 1) adding additional virtual nodes 2) using transformer GNNs
- However, there is disagreement on the best solution for oversquashing within the literature: . some argue transformers can capture long range dependencies more naturally. some argue that adding virtual nodes to an MPNN is equivalent in theory to transformer GNN. some argue that there are no benchmarks in which transformer GNN strongly outperforms MPNN with virtual nodes.

Reconstruct the standard oversquashing narrative + Notice the cracks in the standard story

- The 2 difficulties in training deep MPNNs are oversquashing and oversmoothing

- Here, we focus on oversquashing
- Oversmoothing refers to the phenomenon where feature vectors of neighbouring nodes all converge to the same feature vector (or converge to some small subspace). This is a problem because it means that nodes are indistinguishable. This occurs especially in deep GNNs, where node feature vectors are updated by averaging with their neighbours at each GNN layer.
- It is important to distinguish oversmoothing from oversquashing.
- The term oversquashing, coined by Alon and Yahav [2], refers to the difficulty of training MPNNs on long-range tasks. This is the standard interpretation of oversquashing in the literature.
- According to Alon and Yahav, oversquashing is due to the 'bottleneck effect'
- the 'bottleneck effect' refers to information passing between (distant) nodes being limited by the need for larger dimensionality of feature vectors of intermediate nodes. These intermediate nodes must compress exponentially more information into finite length feature vectors, as the length of the task increases.
- This paper states that the framing of oversquashing as a bottleneck phenomenon is too narrow.
- In fact, since the original 'bottleneck' explanation, several papers have emphasised different explanations for oversquashing.
- In particular, low graph connectivity and vanishing gradients
- And so the single 'label' of oversquashing mixes together several mechanisms (bottleneck effect, vanishing gradients, low graph connectivity)
- Framing oversquashing as any one of these mechanisms is problematic.
- This is because oversquashing is often referred to as refers to the difficulty of training MPNNs on long-range tasks (as coined by Alon and Yahav.

- However, these failure mechanisms may not necessarily be solely related to long-range tasks.
- In particular, the 'bottleneck effect' may be prone to short range tasks.

Extract the paper's central claim from the abstract

- This is the central claim of the paper: the bottleneck phenomenon is not limited to long-range tasks.
- This is surprising because the bottleneck effect has been largely associated with, and studied for long range tasks.
- Furthermore, an additional claim is the vanishing gradient phenomenon, is closely associated with long-range tasks.

Understand the paper's proposed distinction

- bottleneck effect: information from a relatively large number of nodes must still be compressed through relatively few number of intermediate nodes in order to solve the task (regardless of the number of message-passing steps)
- vanishing gradients: task requires many message passing steps, and gradients decay across those repeated steps
- it is useful to separate the variables: **how far information travels** from **how much information must be compressed**
- vanishing gradients is related to the first variable, while bottleneck effect is related to the second variable.

See how the paper positions itself against prior explanations

- Current theories for the bottleneck effect do not explain why the bottleneck effect happens in short range tasks. (as these theories capture long range / gradient decay failures)
- Virtual nodes have been proposed as a solution to oversquashing.
- The intuition for virtual nodes is that they shorten communication paths
- However, adding virtual nodes fail on this new short-range bottleneck case.

- Transformers, on the other hand, do succeed on this new case.
- Self-attention enables interactions between all pairs of nodes in a single step / layer. Therefore, unlike message passing which only allows for interactions between immediate neighbours in a single step, transformers do not rely on (potentially bottleneck) intermediate nodes to pass information.
- A key claim of the authors is that transformers are more compelling to overcome the bottleneck effect than MPNNs (even when using virtual nodes)

Rebuild the logic of Section 1

Understand the structure of the authors' argument

- Existing work shows MPNN+VN can be competitive with transformers.
- But this existing work is performed on messy real-world benchmarks, making the empirical debate unclear.
- And so this paper proposes a clean interpretable test case which isolates exactly the question we want to study: do transformers or MPNN better overcome the bottleneck problem?
- In particular they construct a **family of graph-learning problems**.

Part H — Understand the structure of the authors' argument

- What properties do these tasks / problems have, and why were they chosen?
- The authors construct problems that admit exact solutions with just **two MPNN iterations**.
- They also state that in order to solve these tasks, any MPNN must use very large node-feature dimensions (this is the solution to the bottleneck problem)
- Therefore, this choice of problem family precisely proves that the bottleneck problem can be short-range.
- Remember that low graph connectivity was found as a potential culprit for oversquashing.

- This **family of graph-learning problems also disentangles the bottleneck effect from low connectivity. The bottleneck effect can happen even when there is high connectivity. (Maybe this family of graphs have high connectivity?)**
- They also disentangle the bottleneck effect from vanishing gradients. They show empirically that vanishing gradients do **not** occur for these problems.
- So we have 3 separate mechanisms for failure of learning in MPNNs: low connectivity, bottleneck effect, vanishing gradients.
- (we want to read to make sure that the paper fully distinguishes these)

Part I — Contrast with long-range synthetic tasks

- There are popular synthetic long-range tasks from prior literature (like **Tree Neighbors-Match / Ring Transfer**)
- The authors find that these problems suffer from vanishing gradients but not suffer from the bottleneck effect
- This contradicts Alon and Yahav who explain that the difficulty of long range tasks for MPNN is due to the bottleneck effect.
- Therefore, these long range tasks should be used as a benchmark for gradient decay.

Reach the paper's final framing

- I would say the main contribution is 1) disentangling the MPNN failure mechanisms: bottleneck effect and vanishing gradients. 2) showing that the bottleneck effect (but not vanishing gradient) can occur in short range problems. and showing that for long range problems, we mostly see vanishing gradient, but it is not clear how much they discuss the extent to which bottleneck effect can occur in long range problems 3) showing that transformers beat MPNNs at overcoming the bottleneck effect.

Section 2

▼ mental notes

- 'Due to the permutation invariance of MPNNs, the central nodes are indistinguishable. As a result, adding

more central nodes does not resolve the bottleneck phenomenon, even though it substantially improves the connectivity of the graph.'

- this is incredibly surprising, and I want to understand what exactly is permutation invariance of MPNN, and why this leads to failure of MPNN for the two radius (general k) task

Re-enter the section with the right goal

- The study chooses synthetic graph-transfer problems rather than using messy real datasets because they enable you to easily isolate different failure mechanisms of MPNNs, without confounding.
- A synthetic graph-transfer task should clearly expose whether the failure comes from one of the 3 failure mechanisms.
- All of the studied tasks have central nodes. These are the 'intermediate nodes' which may be the bottleneck, in the 'bottleneck effect'.
- and so it is sensible to isolate these central nodes when studying the bottleneck effect failure mechanism.
- all tasks are such that central nodes' sole purpose is to relay information, rather than provide information from their original node label.

Part B — shared task template

- Each node feature consists of an identifier and a label.
- Identifier defines which node is being referred to, while the label is the information to transfer
- source identifiers must be unique so that the source is uniquely addressable.
- central nodes all get identifier 0 because the model does not need to distinguish them. they all have the same role, which is to pass information.
- target identifiers tell the model from which source that target should receive its label
- the task is about routing the right information, not just aggregating all information

- identifiers and labels encoded as one-hot vectors as these are orthogonal, and avoid ambiguity about representations. but this is a good question.
- these tasks should be the cleanest possible tasks to show the required mechanism failing of MPNN.

Part C — Figure out why Tree Neighbors-Match is the old canonical example

- The graph structure in Tree Neighbors-Match is of a binary tree, with an additional target node attached to the root of the tree.
- The source nodes are the leaves of the binary tree.
- The target node is asked to recover the label of the source node which has the same node identifier as the target node.
- This task is hard for message passing, especially as depth of the binary tree increases, because the number of leaves grow exponentially as depth increases. The root node must therefore summarise information from exponentially more leaf nodes.
- In this Tree Neighbors-Match problem, the length of the Tree Neighbors-Match task is intertwined with the bottleneck effect. As the task length increases, the bottleneck effect becomes more severe because root node must therefore summarise information from exponentially more leaf nodes as depth increases.
- However, the length of the task is not always related to the bottleneck effect.

Part D — Ring Transfer

- The graph structure in Ring Transfer consists of a source and target node, connected by 2 disjoint paths of length r .
- The source is asked to send the target its label.
- The key variable controlling the difficulty of Ring transfer for MPNNs is the length r (which is called the problem radius)
- Theorem 1 states that 'the Ring Transfer task with radius r requires at least r iterations of an MPNN'. and so for large r , Ring Transfer is inherently a long range task.

- Furthermore Theorem 1 states that there exists an MPNN which can solve a Ring transfer task, whose node feature dimension is independent of r .
- Therefore, the Ring transfer task isolates failure mechanisms which are related to depth, and definitely not the bottleneck effect.
- to be devils advocate, perhaps the network has learned to solve the problem by compressing information from many nodes into intermediate nodes (even though theoretically, it does not have to do this to solve the task), and so failure could still be due to bottleneck effect?

Part E — Pause and make the first comparison

- there are two variables at play in these problems.
- **how far** information must travel in order to solve the task (distance travelled)
- **how much** information must be compressed through an intermediate representation to solve the task (amount compressed)
- as we said before, Tree Neighbors-Match problem entangles these variables
- and so this is how the 2 radius problem is motivated. we want a task which isolates these two variables.
- Ring transfer already isolates the distance travelled variable. if a model fails on ring transfer, the failure must be related to the required distance travelled in order to solve the task.
- Two-radius isolates the amount compressed variable. If an MPNN fails at this task, the failure must be related to the amount of information that must be compressed through an intermediate representation, in order to solve the task (and definitely not the distance required)

Part F — Enter the Two-Radius problem with the right lens

- A graph, G_n , in the **Two-Radius family** contains **n source nodes, n target nodes and a single central node. Each source node and each target node is connected to this single source node.**
- The graph distance (radius) from source to target node is only 2.

- Each source node has node feature containing a unique identifier and label.
- Each target node has a node feature containing an identifier.
- Each target must recover the label of the source node which has a matching identifier.
- (This is the same task as in all problems so far).

Part G — See the bottleneck hiding inside the short radius

- To solve the task the central node has to store the identifier and label for every source node, within its representation
- The central node's representation must act like a lookup table in vector form
- It is important that source labels themselves are distinct and target-specific because this makes the bottleneck effect stronger.
- There are $n!$ different ways labels can be assigned to the n source nodes (as identifiers remain fixed, but labels may change).
- The central node must be able to represent every one of these source node label assignments differently.
- Suppose that for two different source node label assignments, the central node output the same representation after 1 step. Then the correct labels could not possibly be assigned to the targets for both cases.
- As the number of source states grows, the number of possible source node label assignments grow, and subsequently the required dimension of the central node representation must increase so that it is possible to produce unique distinguishable representations for every such source node label assignment.

Part H — Read Theorem 2 conceptually, not line-by-line

- Theorem 2 states that there exists an MPNN which can solve the transfer task for $G_{n,k}$ in $T=2$ iterations. But such an MPNN needs a large dimension of intermediate node features.

- Theorem 2 uses a lower bound on $\text{Sum}_t^T d_t$. This is because information can be distributed across iterations. If you use more iterations, T , the constraint on d_t (lets assume d_t is fixed for all t) becomes less strict. In other words, if you increase the number of iterations, the total available representational room of the central nodes increases. Therefore increasing iterations may help overcome the bottleneck effect?
- This lower bound grows with n .
- the bound stated using b -bit floating arithmetic because it formalises finite representational precision. The number of bits, b , per coordinate influences the number of distinguishable states the node's feature vector can be in.
- 'Ring Transfer needs more **iterations/depth**; Two-Radius needs more **feature dimension/width**' but actually the large feature dimensionality requirement in two radius can also be alleviated with more iterations, as theorem 2 shows?

Part I — Why adding more central nodes does not rescue the task

▼ reminder of permutation equivariance and invariance

in the sense that they preserve the adjacency information between the nodes described by the u th and v th rows of $\mathbf{X}$ – cf. Figure 5.1. Therefore, applying a permutation matrix $\mathbf{P}$ to the node features $\mathbf{X}$ automatically implies applying it to $\mathbf{A}$'s rows and columns, $\mathbf{PAP}^\top$.⁶ We say that (a graph-wise function⁷) f is *permutation invariant* if

$$f(\mathbf{PX}, \mathbf{PAP}^\top) = f(\mathbf{X}, \mathbf{A}) \quad (5.49)$$

and (a node-wise function) $\mathbf{F}$ is *permutation equivariant* if

$$\mathbf{F}(\mathbf{PX}, \mathbf{PAP}^\top) = \mathbf{PF}(\mathbf{X}, \mathbf{A}) \quad (5.50)$$

for any permutation matrix $\mathbf{P}$.

- Permutation invariance: regardless of the ordering of nodes, the output should remain the same
 - Permutation equivariance: if you permute the input nodes, the output should permute in the same way.
-
- we can enforce permutation equivariance of function $\mathbf{F}$ by specifying a local function, ϕ , which operates over features of a node and its neighbourhood.

- Then as long as ϕ is permutation invariant, F as defined below will be permutation equivariant.

$$F(\mathbf{X}, \mathbf{A}) = \begin{bmatrix} - & \varphi(\mathbf{x}_1, \mathbf{X}_{\mathcal{N}_1}) & - \\ - & \varphi(\mathbf{x}_2, \mathbf{X}_{\mathcal{N}_2}) & - \\ & \vdots & \\ - & \varphi(\mathbf{x}_n, \mathbf{X}_{\mathcal{N}_n}) & - \end{bmatrix} \quad (5.53)$$

- here is a diagram illustrating the action of ϕ :

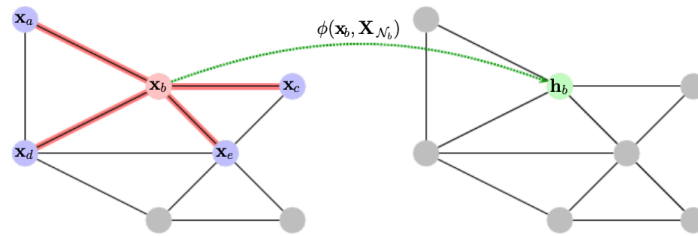


Figure 5.2

An illustration of constructing permutation-equivariant functions over graphs, by applying a permutation-invariant function φ to every neighbourhood. In this case, φ is applied to the features $\mathbf{x}_b$ of node b as well as the multiset of its neighbourhood features, $\mathbf{X}_{\mathcal{N}_b} = \{\mathbf{x}_a, \mathbf{x}_c, \mathbf{x}_d, \mathbf{x}_e\}$. Applying φ in this manner to every node's neighbourhood recovers the rows of the resulting matrix of latent features $\mathbf{H} = F(\mathbf{X}, \mathbf{A})$.

- We now consider GNN layers which are permutation equivariant functions, $F(\mathbf{X}, \mathbf{A})$, specified by a permutation-invariant function, ϕ , which operates over features of a node and its neighbourhood (as discussed above)
- There are 3 main spatial flavours of GNNs
- convolutional, attentional and message passing.

In all three flavours, permutation invariance is ensured by *aggregating* features from $\mathbf{X}_{\mathcal{N}_u}$ (potentially transformed, by means of some function ψ) with some permutation-invariant function $\oplus$, and then *updating* the features of node u , by means of some function φ . Typically,¹³ ψ and φ are learnable, whereas $\oplus$ is realised as a nonparametric operation such as sum, mean, or maximum, though it can also be constructed e.g. using recurrent neural networks with appropriate constraints (R. L. Murphy et al. 2018).

Finally, the **message-passing** flavour (Battaglia et al. 2016; Gilmer et al. 2017; Battaglia et al. 2018) amounts to computing arbitrary vectors ("messages") across edges,

$$\mathbf{h}_u = \varphi \left(\mathbf{x}_u, \bigoplus_{v \in \mathcal{N}_u} \psi(\mathbf{x}_v, \mathbf{x}_v) \right). \quad (5.56)$$

- To ensure that ϕ is permutation invariant, the aggregation function is chosen to be e.g. sum (can also be mean or max or min).

- to summarise, permutation equivariance of an MPNN layer, $F(\mathbf{A}, \mathbf{X})$ is enforced by ensuring that F is a function of the form:

$$\mathbf{F}(\mathbf{X}, \mathbf{A}) = \begin{bmatrix} \text{---} & \varphi(\mathbf{x}_1, \mathbf{X}_{\mathcal{N}_1}) & \text{---} \\ \text{---} & \varphi(\mathbf{x}_2, \mathbf{X}_{\mathcal{N}_2}) & \text{---} \\ & \vdots & \\ \text{---} & \varphi(\mathbf{x}_n, \mathbf{X}_{\mathcal{N}_n}) & \text{---} \end{bmatrix} \quad (5.53)$$

- where ϕ is permutation invariant.
- Remember that there are 3 failure mechanisms at play: bottleneck effect, low connectivity and vanishing gradients
- Theorem 2 proves that for Two radius, as n grows, the feature dimension of intermediate nodes must also grow. This is the bottleneck effect.
- However, it is also true that the simple Two radius problem has low connectivity since all source nodes are only connected to a single central node.
- one might therefore argue that the Two radius problem entangles low connectivity and the bottleneck effect.
- if an MPNN fails on two-radius, it is not clear which of these 2 failure mechanisms to tackle.
- Therefore, the generalised two-radius problem was introduced to isolate solely the bottleneck effect, disentangling it from low connectivity.
- The more general graph family $G_{n,k}$ now additionally consists of k central nodes, and each is connected to all sources and all targets.
- The information routing problem is unchanged. The task still asks the target nodes to recover the label of the source node with matching identifier

- One might expect that increasing the number of central nodes will alleviate the bottleneck problem in MPNNs because the representational capacity of several central nodes should be greater than a single central node.
- However, because MPNNs are permutation equivariant, and the k central nodes are structurally identical, this is not the case.
- In particular, remember that, in any MPNN layer, ϕ must be permutation invariant. then note that each central node has the same representation vector, and same set of neighbours (all source nodes and target nodes). Therefore, ϕ must produce the same output for all central nodes.
- And so after the first MPNN layer, the central nodes still have the same node representation, and so will continue to have the same node representation across all layers of the MPNN. In other words, the node representations of the central nodes evolve identically across message passing steps.
- This tells us that improving graph connectivity, may not relieve the bottleneck effect for a permutation equivariant model (such as an MPNN).
- Graph structure alone does not determine the bottleneck effect. The symmetry within the graph also is important.
- If the central nodes, for example, had different initial node representations, then the ϕ function is not restricted to output the same value for each central node, and so the MPNN could take advantage of the representational capacity of all central nodes.

Part J — Understand the practical remark after Theorem 2

- It is important to state that Theorem 2 is a theorem regarding the representational capacity of MPNNs.
- It states that given the MPNN layer has node feature dimension which is large enough, an MPNN can solve the task in 2 iterations. (i.e. there exists some parameter configuration such that the MPNN can solve the task in 2 iterations)
- However, the theorem says nothing regarding the ease of SGD finding this parameter configuration reliably during model training.

Part K — Lightly read 2.2 for the punchline

- The authors ask several empirical questions:
- 1) For the the 2 radius task, theorem 2 states that an MPNN will be limited by the bottleneck effect. Is this true empirically?
- The answer this question by determining if when keeping node representation length fixed, but increasing n , the task performance decreases. If yes, this is good evidence for the bottleneck effect. They test several standard MPNN models to ensure the empirical claim holds across the MPNN family
- 2) Theoretically, the generalised 2 radius task, should not be easier than the standard 2 radius, because of the permutation equivariance of MPNNs. Is this true empirically?
- They answer this question by observing task performance while increasing number of central nodes k (holding everything else fixed)
- 3) We expect that a set transformer is not limited by the bottleneck effect. Is this true empirically?
 - the Set Transformer ignores the graph edges and treats the node features as a multiset
 - Every node can attend to every other node directly.
 - A target node can, in principle, attend directly to the source node with the matching identifier and copy its label, without waiting for this label information to be routed through the central nodes.
- 4) We expect an MLP is not limited by the bottleneck effect, but fails because of poor inductive bias for the task.
- For 1), they find that GAT is the strongest MPNN. With high hidden dimension, it can reach around 90% accuracy, even with $n=200$. However, GAT needed a very low learning rate and many epochs. The transformer, in comparison, requires around 10 epochs. This implies that the transformer architecture's inductive biases are better suited for this two-radius task.

- The results for 3) imply that for the 2 radius problem, considering graph structure is an unhelpful inductive bias

Section 3-5

- Section 2 shows that MPNN fails at 2-radius task, and this failure is likely due to the bottleneck effect mechanism.
- But it may be that existing methods to characterise or measure oversquashing can already capture this bottleneck effect mechanism.
- We now want to show that the most common methods to characterise or measure oversquashing fail to capture the bottleneck effect mechanism in 2 radius.

Part B — Problem radius and vanishing gradients

- The initial method for characterising oversquashing is large problem radius. Since 2-radius has a radius of 2, this characterisation fails to predict the bottleneck mechanism in 2 radius.
- Following this, one might predict that the underlying issue is not the radius itself but vanishing gradients. And perhaps vanishing gradients can still happen even in short range tasks, such as 2-radius.
- However, empirically they find that gradients do vanish as problem radius r increases for ring transfer, but this is not the case when we increase n in two-radius
- Gradient decay theorems depend on large distance between source and nodes (or large number of message passing steps required)
- In 2-radius, certainly the distance required is small (only 2). For the number of message passing steps, the limitation is less clear. as message passing steps increase, the node representation length requirement of the central nodes decrease.

Part C — Topological and spectral explanations

- A broader family of explanations for oversquashing are graph-structural explanations

- spectral gap, connectivity, Ricci curvature, effective resistance, and related ideas
- these all attempt to characterise oversquashing through graph topology
- For the generalised two-radius problem, graph topology is misleading because you can increase connectivity, but as long as the central nodes have the same initial node representation, the bottleneck effect is not changed in theory (due to permutation equivariance of MPNNs)
- The bottleneck arises due to the interaction between the graph, the node representations, the task, and the architectural symmetry constraints of MPNNs.
- The Ricci-curvature metric is especially common for characterising oversquashing, but fails for 2-radius.
- We have learnt that most explanations for oversquashing are related to task-length, vanishing gradients and poor connectivity, and miss the bottleneck effect

Part D — Why Section 4 matters: virtual nodes

- Virtual nodes are a strategy for MPNNs to overcome oversquashing. The intuition for their utility is that they reduce the problem radius and improve connectivity. And so failure mechanisms related to either of these will be alleviated by virtual nodes.
- This is not the case for 2-radius.
- However, virtual nodes may be chosen so that their node representations are distinguishable from the central nodes (and each other).
- And so the MPNN is able to utilise these virtual nodes for storing information (unlike additional central nodes).
- The authors test adding a single central node, and find that across varying node count n , GCN + VN only marginally improves failure of GCN on 2-radius.
- Perhaps additional virtual nodes may help.

Part E — Why transformers matter here

Additional insights for the write up

Symmetry in graph structure of ring transfer

- Just how there is symmetry of central nodes in two-radius
- there is symmetry of the two paths from source to target in ring transfer
- the two paths are structurally identical
- the MPNN (which is permutation equivariant) will compute the exact same hidden representations for the corresponding nodes on both paths (assuming they have the same node identifiers).
- If you add more identical paths, to create a multi-ring, the same will be true. the corresponding nodes in each path will evolve in the same way.
- You may then ask why have two separate paths, rather than one?
- This is because (?)

What is a virtual node?

- A Virtual Node (VN) is an artificial node added to the graph that is connected to every single real node in the graph.
- Any node can therefore reach any other node via the virtual node
- The "Single-VN variant" is as follows
 - at layer 0, the VN is just an all-zero vector.
 - At layer l, after message passing between all real nodes has taken place, the VN aggregates the node features of all nodes in the graph (via sum or mean)
 - this aggregated node vector is passed through an MLP, before being added to the feature vector of every real node in the graph
- for the "Multiple-VN variant", each VN has its own MLP and is updated independently. At the end of each layer, the embeddings of all virtual nodes are added to each real node.

Will adding a virtual node help in ring transfer problem?

Yes. The original problem with Ring Transfer was that the distance r was very large. The signal died because it had to pass through r layers of weight multiplications (W^r).

By adding a Virtual Node, the distance from the Source to the Target is now 2. The signal only requires 2 layers to reach the target. The gradient will no longer vanish.

- however adding a virtual node, means that the Virtual Node has to aggregate information from all $2r$ nodes in the ring simultaneously
- and so adding the virtual node creates a two-radius problem (or a very similar problem)
- and so we may have a bottleneck effect

What tasks do Set Transformers fail on, but MPNNs succeed on? What succeeds at both?

- Set transformers would fail at topological tasks
- e.g. given an input graph, output whether that graph is a ring or a star
- if a ring and a star (with the same number of nodes) has identical node representations, then a set transformer would give identical outputs for both
- graph transformers overcome the limitations of both set transformers and MPNNs.

What is a graph transformer

▼ set transformer reminder

- A **Set Transformer** is a Transformer adapted for **unordered sets** instead of sequences
- Self-attention layers act on a set rather than a sequence, with no casual masking applied, such that all nodes can attend to one another.
- Furthermore, Set transformers are permutation-equivariant. Permuting the input elements just permutes the intermediate outputs of each layer. Positional encodings are therefore not used for set transformers.

- If it is a graph-level task rather than node-level, then the final pooled output can be made permutation-invariant.

▼ MPNN reminder

Finally, the **message-passing** flavour (Battaglia et al. 2016; Gilmer et al. 2017; Battaglia et al. 2018) amounts to computing arbitrary vectors (“messages”) across edges,

$$\mathbf{h}_u = \varphi \left(\mathbf{x}_u, \bigoplus_{v \in \mathcal{N}_u} \psi(\mathbf{x}_u, \mathbf{x}_v) \right). \quad (5.56)$$

- The key distinction between set transformer and MPNN is that in a set transformer, all nodes interact with each other, whereas in an MPNN, nodes only interact with their immediate neighbours. Both are equivariant to node permutation, but MPNNs take advantage of graph structure.
- The goal of a graph transformer is to enable each node to communicate directly to every other node, but to modify this communication based on the structure of the graph
- A graph transformer achieves this by altering the attention mechanism
- In standard self-attention (as used in set transformers), $\text{attn}(u,v)$ computes how much node u should attend to node v using only the node representations of u and v (and not based on their connectivity).

$$\text{Attention}(u, v) = \text{Softmax} \left(\frac{(W_Q \mathbf{x}_u)^\top (W_K \mathbf{x}_v)}{\sqrt{d}} \right)$$

- A graph structure extends attention in two ways 1) positional encodings 2) Structural Encodings/Biases
- positional encodings:
 - Before the first layer, you compute a positional encoding for each node and add (or concatenate) it to the node's initial features

- a common way to compute PEs are via laplacian eigenvectors.
- Structural Biases
 - this method forces the attention score to change based on the distance between u and v in the graph

$$\text{Attention}(u, v) = \text{Softmax} \left(\frac{(W_Q \mathbf{x}_u)^\top (W_K \mathbf{x}_v)}{\sqrt{d}} + \mathbf{Bias}(u, v) \right)$$

- a common way to compute bias is via shortest path distance between nodes u and v . For such a shortest path distance, you look up a learnable scalar. This technique was popularised by Graphormer.
- A graphormer can be written as the message passing template too

$$\mathbf{h}_u = \varphi \left(\mathbf{x}_u, \bigoplus_{v \in \mathcal{N}_u} \psi(\mathbf{x}_u, \mathbf{x}_v) \right)$$

▼ details

1. **The Neighborhood ($\mathcal{N}_u$) becomes the Entire Graph (V):** Instead of only summing over connected neighbors, the sum $\bigoplus$ is taken over *all nodes* in the graph.
2. **The Message Function (ψ) becomes Attention:** The function $\psi(\mathbf{x}_u, \mathbf{x}_v)$ is replaced by the attention mechanism. It calculates a message from v , multiplies it by the attention score (which now includes Structural Biases), and sends it to u .

Graph transformer vs MPNN

- An MPNN inherently "knows" the graph structure because it only passes messages along real edges. MPNN also encodes the assumption that neighbours are important (homophily)

- Positional/Structural Encodings do help a graph transformer see the graph structure, but this may not be perfect.
- MPNNs scale linearly with the number of edges, $O(|E|)$ (and because real world edges are usually sparse, E is close to N)
- Transformers compute attention between every pair of nodes, meaning they scale quadratically $O(N^2)$.
- Using a graph transformer is therefore impossible on large scale node classification. You can't fit transformer on e.g. 5 million node graph. MPNNs like GraphSAGE can sample neighbours of large graphs, which makes them more scalable.
- While MPNNs suffer from oversmoothing (unlike transformers), for homophilic tasks (i.e. neighbours are similar), the smoothing effect of MPNNs is actually useful. e.g. on citation networks like CORA).

What are the different classes of models for graph data?

▼ summary

A. Spectral & Wavelet GNNs (e.g., LR-GWN)

- *How they work:* Instead of passing messages along edges (Spatial domain), they convert the graph into "frequencies" using the Eigenvectors of the Graph Laplacian (Spectral domain).
- *Why they are good:* They can achieve global, long-range vision without the $\mathcal{O}(N^2)$ cost of Transformers. (This is exactly what the *Long-Range Graph Wavelet Networks* paper you selected earlier does!).

B. Subgraph GNNs / Higher-Order GNNs (e.g., ESAN)

- *How they work:* The course teaches you that MPNNs are limited by the 1-Weisfeiler-Lehman (1-WL) test. Subgraph GNNs break this limit. Instead of passing messages between *nodes*, they extract a subgraph (e.g., a "bag of subgraphs") for each node and pass messages *within* and *between* these subgraphs.
- *Why they are good:* They are strictly more expressive than standard MPNNs and can count complex structures like triangles and cycles (which standard MPNNs cannot do).

C. Topological Deep Learning (Simplicial / Cellular Neural Networks)

- *How they work:* Graphs only have nodes (0D) and edges (1D). Topological models allow for "faces" (2D triangles) and "volumes" (3D pyramids). Messages are passed not just node-to-node, but edge-to-edge and triangle-to-triangle.
- *Why they are good:* Excellent for modeling complex, multi-body interactions (like a chemical ring where 6 atoms act as a single unit, rather than 6 pairwise edges).

D. Continuous Graph Neural Diffusion (Neural PDEs)

- *How they work:* They treat message passing not as discrete layers, but as a continuous flow of heat or fluid over time, modeled using Differential Equations (ODEs/PDEs).
- *Why they are good:* They provide elegant mathematical solutions to the "Oversmoothing" problem by altering the diffusion equations to act like oscillators rather than purely dissipating heat.

- For 3D Physics & Chemistry (e.g., Molecular Dynamics, AlphaFold), SE(3)-Equivariant GNNs (in particular MPNNs) are the best here. SE(3) is symmetry group of 3D rotations and translations. Graph Transformers fail here unless they are heavily modified to be SE(3)-equivariant
- For Discovering Long-Range Patterns in small/medium graphs, the best is Graph Transformers (with PE/SE) or Hybrid Models (like LR-GWN).
- For standard Social/Citation Networks, the best is Standard MPNNs (GAT, GraphSAGE, GCN) or Rewired MPNNs (if dealing with heterophily or bottlenecks).

What class of model might work for Two-radius, but is not as computationally expensive as a (graph) transformer?

- **Spectral Filters / Wavelet GNNs e.g. LR-GNN**
- **subgraph GNNs (e.g., ESAN, GSN)**